

Servicio: kpi_operativo.py

Archivo: `services/kpi_operativo.py` · Documento técnico-pedagógico · Generado 2026-05-20 13:46

Es el módulo de monitoreo: recoge cada atención cerrada, cada error capturado y los convierte en indicadores (KPIs) que permiten detectar cuellos de botella, áreas saturadas y debilidades del flujo.

Resumen general

El cambio fue total. El módulo original calculaba métricas básicas a partir del estado momentáneo de las colas. Ahora lleva un **historial completo** de cada atención y cada error, con timestamp, y aplica criterios (umbrales) para detectar problemas reales del flujo.

ANTES	AHORA
Solo había mostrar_kpi() que devolvía texto	Estado interno: atenciones = [], errores = [] Funciones para registrar, calcular y reiniciar.
Las métricas se basaban en el estado actual de la cola (al cerrar nada quedaba)	Cada atención cerrada queda registrada con hora, área, paciente, cita y empresa.
No había concepto de 'debilidad detectada'	identificar_debilidades() aplica umbrales (saturación, espera máxima, errores...)
No diferenciaba por empresa, ni mostraba citas atendidas vs en flujo vs programadas	Reporte segmentado: por empresa, por área, por estado de cita.

1. El estado del KPI

Dos listas globales son el corazón del módulo. Una guarda cada atención cerrada (con todos sus datos), la otra guarda cada error capturado (validaciones fallidas, intentos inválidos, etc.). Todo con timestamp.

```
atenciones = [] # lista de eventos de atención cerrada
errores = []    # lista de errores capturados

def registrar_atencion(area, paciente, tiempo_espera_min, cita=None):
    atenciones.append({
        "area": area,
        "paciente": paciente,
        "tiempo_espera": tiempo_espera_min,
        "timestamp": datetime.now(),
        "cita_id": cita.id_cita if cita else None,
        "empresa": cita.paciente.empresa if cita else None,
        "tipo_examen": cita.emo.tipo if cita else None,
        "completado": (cita is not None and not cita.areas_pendientes),
    })

def registrar_error(origen, mensaje):
    errores.append({
        "origen": origen,          # "flujo", "citas", "ui"...
        "mensaje": mensaje,
        "timestamp": datetime.now(),
    })
```

Para principiantes: Guardar el *timestamp* (momento exacto) de cada evento es lo que permite luego construir el timeline del paciente o calcular cuánto tardó cada paso del examen. Sin esta información, solo tendríamos números agregados.

2. calcular_kpi() — el panel de indicadores

Esta función combina los datos del estado actual de las colas con el historial de atenciones para devolver un diccionario gigante con todos los indicadores que la UI necesita mostrar.

```
def calcular_kpi():
    estimado = _tiempo_estimado_por_area()    # cuánto tardará la cola actual
    atendido = _tiempo_atendido_por_area()    # cuánto se acumuló ya atendido

    return {
        "total_citas": total_citas,
        "citas_programadas": citas_programadas,
        "citas_en_flujo": citas_en_flujo,
        "citas_atendidas": citas_atendidas,
        "total_atenciones": total_atenciones,
        "total_errores": total_errores,
        "tiempo_total_cola": tiempo_total_cola,
        "promedio_espera_cola": promedio_cola,
        "promedio_espera_atendido": promedio_atendido,
        "max_espera_atendido": max_atendido,
        "area_cuello_botella": area_cuello,
        "areas_saturadas": saturadas,
        "tiempo_estimado_por_area": estimado,
        "tiempo_atendido_por_area": atendido,
        "atenciones_por_area": conteo_at,
        "por_empresa": _por_empresa(),
    }
```

3. identificar_debilidades() — detección automática

Es donde el sistema empieza a 'razonar'. Se definen umbrales (porcentajes y valores límite) y se compara contra los KPIs. Si algo los supera, se agrega a la lista de debilidades que se muestra al usuario.

```
UMBRAL_ESPERA_MIN = 30
UMBRAL_SATURACION_PCT = 80

def identificar_debilidades():
    kpi = calcular_kpi()
    debilidades = []

    # ¿Alguna área superó el 80% de capacidad?
    for area in flujo.areas:
        ocupacion_pct = (len(flujo.areas[area]) / flujo.LIMITE) * 100
        if ocupacion_pct >= UMBRAL_SATURACION_PCT:
            debilidades.append(
                f"Área '{area}' al {ocupacion_pct:.0f}% de su capacidad..."
            )

    # ¿Cuál es el cuello de botella ahora mismo?
    if kpi["area_cuello_botella"]:
        debilidades.append(f"Cuello de botella: '{kpi["area_cuello_botella"]}'...")

    # ¿Alguna espera superó los 30 min?
    if kpi["max_espera_atendido"] > UMBRAL_ESPERA_MIN:
        debilidades.append("Espera máxima supera el umbral...")

    return debilidades or ["Sin debilidades detectadas con los umbrales actuales."]
```

Para principiantes: Los **umbrales** son números que separan 'normal' de 'problema'. Aquí están escritos como constantes al inicio del archivo para que sean fáciles de ajustar. Si un día decides que 30 minutos de espera son aceptables, solo cambias `UMBRAL_ESPERA_MIN = 45` y el sistema entero se adapta.